
RELAGRAM: Recursive Latent Graph Model for the Capacitated Vehicle Routing Problem

Vladimir Kirill Bickov

Delft University of Technology, The Netherlands
me@nevolodia.com

Neil Yorke Smith

Delft University of Technology, The Netherlands
n.yorke-smith@tudelft.nl

Abstract

Latent recursion repeatedly processes internal representations through the same network. Recent work has shown this approach to be effective on structured reasoning tasks such as Sudoku and maze solving. However, whether latent recursion can support neural routing remains unclear. We introduce RELAGRAM (Recursive Latent Graph Model), a novel approach that brings latent recursion to graph combinatorial optimization. RELAGRAM encodes a capacitated vehicle routing problem (CVRP) instance as edge tokens; a compact Transformer recurrently updates their latent representations. An output head maps the answer state to an edge heatmap, from which an autoregressive decoder constructs a feasible solution. On CVRP-100, the 0.53M-parameter RELAGRAM achieves optimality gaps of 13.4%, 9.8%, and 8.3% under greedy decoding, eightfold symmetry augmentation, and 256-candidate search, respectively. Trained only on CVRP-100, it generalizes zero-shot to CVRP-500 and CVRP-1000, achieving gaps of 15.6% and 21.7% under 256-candidate search. Together, these results demonstrate that latent recursion is a viable approach to neural routing.

1 Introduction

Solving a combinatorial optimization problem in a single, end-to-end pass of a neural network is an inherently difficult. A learned model must navigate a constrained and exponentially large solution space with limited computation. Increasing network depth provides more computation, but also raises training and inference costs [1].

Neural routing methods commonly construct routes autoregressively or iteratively refine complete solutions [2–4]. Both approaches simplify the task by breaking the solution process into series of simpler decisions. However, this simplicity comes at the cost of an information bottleneck. If only the updated solution is carried between iterations, information that exists only in the latent state, such as intermediate reasoning, is lost. This creates a form of representational “amnesia” that makes it difficult to accumulate a richer understanding of the problem.

Latent recursion avoids this bottleneck by carrying the internal state, rather than the candidate solution, between iterations. A solution can be decoded at any stage but is not fed back into the model: each subsequent stage continues directly from the preceding latent state. Every depth therefore produces both an intermediate computation and a valid solution point. Recent latent-recursive models perform strongly on Sudoku, maze solving, and ARC-AGI [5].

However, whether latent recursion transfers from supervised reasoning to neural routing remains unclear. This setting learns directly from route-level rewards, ensures feasibility during decoding, and provides a natural framework for studying generalization across graph sizes and recursion depths.

We introduce RELAGRAM, a novel application of the latent recursion idea to graph combinatorial optimization, and demonstrate it on CVRP. After brief training, a 0.53M model reaches a 13.4% greedy gap on CVRP-100 from a single decoded route, reducing it to 8.3% under a 256-candidate search.

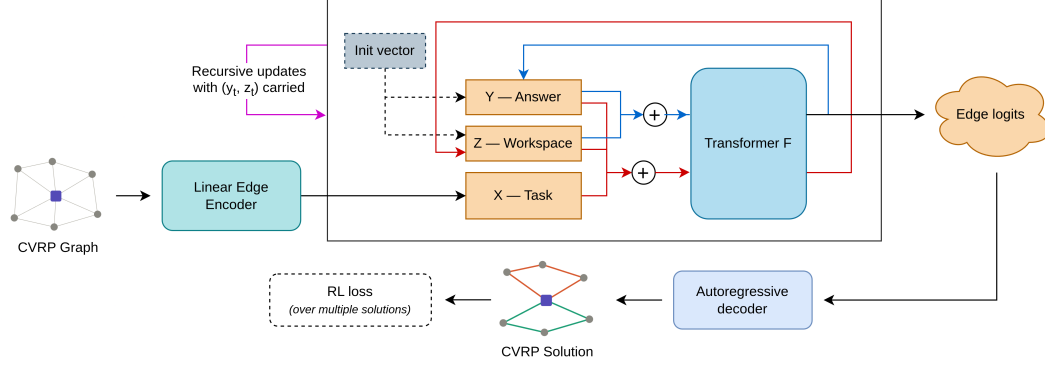


Figure 1: RELAGRAM. A CVRP instance is encoded as edge-token input x ; the answer state y and workspace z start from fixed vectors. The shared Transformer first updates z from $x + y + z$ (red), then updates y from $y + z$ (blue). Both states (y, z) are carried forward (magenta) in a recursive loop. A linear head maps y_t to edge logits, which are decoded into a solution.

On CVRP-1000, the model achieves gaps of 29.9% and 21.7% under greedy and 256-candidate decoding, respectively. Notably, RELAGRAM continues to improve beyond its trained depth: on CVRP-1000 it achieves its best greedy gap of 28.45% at depth ten — six stages deeper than seen during training.

2 Background

Latent recursion. Latent recursion applies the same network repeatedly to an internal state. The Tiny Recursive Model (TRM) keeps two states: a solution state y and a latent reasoning state z . One cycle refines z over repeated passes conditioned on the input and on y , then updates y from the refined z ; a supervision step chains several such cycles, all but the last without gradients. Deep supervision gives every step its own loss, and the states are detached between steps, so the model learns successive corrections without backpropagating across them. TRM achieves strong results on Sudoku, maze solving, and ARC-AGI benchmarks [5].

Edge heatmaps. An edge heatmap records the model’s confidence that each transition belongs to a high-quality solution. DIFUSCO produces such heatmaps for the travelling salesman problem by iteratively denoising edge variables [6]. However, DIFUSCO’s learned edge representation is separate from the procedure that converts it into a discrete solution.

Feasible decoding. A heatmap is not itself a route. For CVRP, each decision must account for the visited customers and remaining vehicle capacity. The Attention Model, a neural routing model, constructs a route autoregressively, selecting one node at a time while masking infeasible actions [2]. DeepACO instead learns a heuristic measure for an ant colony optimization system: a population of simulated ants builds routes in parallel, each choosing only feasible transitions [7].

Learning from sampled routes. REINFORCE allows a routing policy to learn without optimal labels: complete routes are sampled, scored by their cost, and used to improve the policy [2, 8]. The signal is sparse, however, because one route-level reward must guide every edge decision made during construction. Self-imitation adds denser feedback: the lowest-cost sampled route serves as a temporary teacher, and a cross-entropy loss trains the heatmap to favour its transitions [9].

3 RELAGRAM

RELAGRAM brings latent recursion to routing. Between depths it passes its internal state rather than the route itself, so intermediate reasoning is never discarded (Figure 1).

Directed edge representation. RELAGRAM encodes a CVRP instance as sparse directed edge tokens. Each customer i connects to the depot and its $k - 1$ nearest customers, while the depot

connects to every customer; opposite directions are encoded independently. For each edge $e = (i, j)$, the encoder collects nine features: the coordinates, demands, and depot indicators of i and j , and their Euclidean distance. A bias-free linear layer projects them to $x_e \in \mathbb{R}^d$ and scales by $\sqrt{d}$.

Recursion over edge tokens. RELAGRAM maintains the same two states per edge: an answer state y_t and a workspace z_t . They begin as fixed random vectors $y_0, z_0 \in \mathbb{R}^d$. The network F_θ is a two-block Transformer, each block applying self-attention and SwiGLU [10] sublayers with post-norm residuals and RMSNorm [11] without learned scale, as in TRM. At each depth, it updates the workspace and then the answer state:

$$z_t = F_\theta(z_{t-1} + y_{t-1} + x), \quad y_t = F_\theta(y_{t-1} + z_t). \quad (1)$$

A bias-free linear head maps each $y_{t,e}$ to $H_{t,ij}$, the score of moving from i to j . Edges that are not present in sparse-encoded tokens receive logit -10 , providing a fallback when every encoded move is masked.

Route decoding. Starting at the depot, the decoder constructs a route one vertex at a time. At each vertex, the corresponding row of H_t scores every candidate move; visited and capacity-infeasible customers are masked out. Stochastic decoding samples from the remaining logits; greedy decoding takes the largest. The heatmap stays fixed throughout construction.

Learning and the self-imitation auxiliary. At each training depth t , RELAGRAM draws $S = 16$ routes from the masked autoregressive policy. Route $\tau_{t,s}$ receives reward $R_{t,s} = -C(\tau_{t,s})$; their mean $\bar{R}_t = S^{-1} \sum_s R_{t,s}$ is the same-instance REINFORCE baseline [8]. The loss is

$$\mathcal{L}_t = \underbrace{-\frac{1}{S} \sum_{s=1}^S (R_{t,s} - \bar{R}_t) \log \pi_\Theta(\tau_{t,s} | H_t)}_{\text{REINFORCE}} + \underbrace{\lambda_{\text{SI}} \mathcal{L}_{\text{edge}}(H_t, \tau_t^{\text{elite}})}_{\text{Self-imitation}}. \quad (2)$$

REINFORCE raises the likelihood of routes shorter than the same-instance mean. Self-imitation pulls the unmasked heatmap toward the shortest sampled route τ_t^{elite} using cross-entropy loss. We set $\lambda_{\text{SI}} = 0.3$; Θ collects all trainable parameters. Every recursion depth receives one optimizer update, after which its latent states are detached and carried forward, so gradients never propagate down the recursive chain.

4 Experiments

Experimental design. We train on CVRP-100 in the RL4CO CVRPE_{ENV} [12], whose generator samples fresh unit-square instances for every batch. We use a neighbour mask of $k = 20$. The model has 0.53M parameters and is supervised at four depths. Runs use a batch size of 16 with 16 sampled routes per instance, which across the four supervised depths yields 1,024 routes per batch. We evaluate RELAGRAM under two training budgets: 40,000 optimizer updates with a training time of 2.98 hours to compare RELAGRAM with DeepACO; and 160,000 optimizer updates with a training time of 11.94 hours to assess performance and its scaling under a larger training budget. Training uses a 10 GB A100 MIG GPU slice. DeepACO is chosen as our baseline because it is a competitive neural combinatorial optimizer with a similar design: a learned edge heatmap guides a separate solution search. We train it as a 0.52M-parameter model with the same $k = 20$ neighbour mask and optimizer updates matched to RELAGRAM’s 40,000. The models use similar numbers of training trajectories (9.60M for DeepACO and 10.24M for RELAGRAM), but the comparison is not compute-matched: DeepACO sees twice as many base graphs and takes 1.87 times longer to train. Every model is evaluated on the same 256 fixed instances per size. References are PyVRP [13] under per-instance time limits of 5 s at $n = 100$, 20 s at $n = 500$, and 40 s at $n = 1000$. We evaluate three decoding configurations: greedy (one route), eightfold symmetry augmentation, and the 256-candidate combination of symmetry, per-step latent noise [14], and forced different starts.

Solution quality across graph sizes. We first evaluate RELAGRAM at 160,000 optimizer updates on CVRP-100, then apply it unchanged to graphs up to ten times larger. The model reaches an 8.30% gap on CVRP-100 under 256-candidate decoding (Table 1). Without retraining, the same

Table 1: Gap to the PyVRP reference (%; lower is better). For RELAGRAM, Combined256 uses symmetry, noise, and starts; DeepACO uses 16 ants for 16 iterations.

	decoder	RELAGRAM 160k	RELAGRAM 40k	DeepACO 40k
CVRP-100	greedy	13.40	15.55	11.61
	aug8	9.77	11.30	9.16
	combined256	8.30	9.57	6.08
CVRP-500	greedy	22.55	24.78	19.88
	aug8	17.61	19.30	17.68
	combined256	15.64	17.11	17.38
CVRP-1000	greedy	29.91	33.24	23.91
	aug8	24.36	27.07	22.09
	combined256	21.65	24.32	23.31

Table 2: Gap to the PyVRP reference (%; lower is better) across recursion depths under greedy decoding for RELAGRAM at 160,000 optimizer updates. The model is trained at depth four.

	depth							best
	1	2	4	6	8	12	16	
CVRP-100	15.29	13.69	13.40	13.35	13.75	14.69	15.49	13.35 @ 6
CVRP-500	25.46	23.74	22.55	22.36	22.44	21.96	22.80	21.81 @ 11
CVRP-1000	33.21	31.58	29.91	28.80	28.75	28.68	29.12	28.45 @ 10

model reaches gaps of 15.64% and 21.65% on CVRP-500 and CVRP-1000, respectively, under 256-candidate decoding. On CVRP-1000, expanding one greedy decode to 256 candidates improves the gap from 29.91% to 21.65%. The decoder therefore offers test-time scaling that strengthens with instance size, worth 5.10 points on CVRP-100 and 8.26 on CVRP-1000.

Comparison with DeepACO. We next compare RELAGRAM with DeepACO after training both for the same 40,000 optimizer updates. DeepACO is stronger in eight of the nine size-decoder regimes: it achieves 3.94-point and 3.49-point lower gaps on CVRP-100 under greedy and 256-candidate decoding, and a 9.33-point lower greedy gap on CVRP-1000. These results show that RELAGRAM does not yet achieve solution quality competitive with DeepACO. Under 256 candidates, the two converge as instances grow: ants sample every step, so DeepACO’s candidate scaling weakens with size while RELAGRAM’s does not.

Recursion as refinement and search. RELAGRAM improves from its first stage to the trained fourth stage at every size (Table 2). Progress is non-monotonic: a later stage may worsen before producing a better route, so the deepest output is not always the strongest. Applying the loop further yields gains that are negligible at the trained size but grow with instance size: the best gap beyond depth four improves on the depth-four gap by 0.05, 0.74, and 1.46 points on CVRP-100, CVRP-500, and CVRP-1000 at depths six, eleven, and ten, respectively.

5 Discussion and Conclusion

RELAGRAM shows that a small latent-recursive model learns to route. It transfers without retraining to graphs up to ten times larger and can improve beyond its trained depth, suggesting promising inference-time depth scaling. The model is limited, however: it does not reach state-of-the-art results, and the recursion, while useful, does not yield decisive gains. We therefore see RELAGRAM as a proof of concept for applying latent recursion to graph combinatorial optimization.

We hypothesize that an important limitation is the sparse RL signal, which needs further engineering. We are also interested in exploring other architectures, such as pointer networks [15].

Lastly, RELAGRAM’s recursive nature may be very useful for dynamic problems. The loop already runs zero-shot past the conditions it was trained under; if it handles an updated instance as well as extra depth, a single model with a decoder could solve such problems end-to-end, by recurrently processing the last solution and the updated problem.

References

- [1] Yoshua Bengio, Andrea Lodi, and Antoine Prouvost. Machine learning for combinatorial optimization: A methodological tour d’horizon. *European Journal of Operational Research*, 290(2):405–421, 2021. doi: 10.1016/j.ejor.2020.07.063. 1
- [2] Wouter Kool, Herke van Hoof, and Max Welling. Attention, learn to solve routing problems! In *International Conference on Learning Representations*, 2019. 1, 2
- [3] Xinyun Chen and Yuandong Tian. Learning to perform local rewriting for combinatorial optimization. In *Advances in Neural Information Processing Systems*, volume 32, pages 6278–6289, 2019.
- [4] Yaoxin Wu, Wen Song, Zhiguang Cao, Jie Zhang, and Andrew Lim. Learning improvement heuristics for solving routing problems. *IEEE Transactions on Neural Networks and Learning Systems*, 33(9):5057–5069, 2022. doi: 10.1109/TNNLS.2021.3068828. 1
- [5] Alexia Jolicoeur-Martineau. Less is more: Recursive reasoning with tiny networks. arXiv preprint arXiv:2510.04871, 2025. 1, 2
- [6] Zhiqing Sun and Yiming Yang. DIFUSCO: Graph-based diffusion solvers for combinatorial optimization. In *Advances in Neural Information Processing Systems*, volume 36, 2023. 2
- [7] Haoran Ye, Jiarui Wang, Zhiguang Cao, Helan Liang, and Yong Li. DeepACO: Neural-enhanced ant systems for combinatorial optimization. In *Advances in Neural Information Processing Systems*, volume 36, 2023. 2
- [8] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8:229–256, 1992. doi: 10.1007/BF00992696. 2, 3
- [9] Jonathan Pirnay and Dominik G. Grimm. Self-improvement for neural combinatorial optimization: Sample without replacement, but improvement. *Transactions on Machine Learning Research*, 2024. URL <https://openreview.net/forum?id=agT8ojoH0X>. 2
- [10] Noam Shazeer. GLU variants improve transformer. arXiv preprint arXiv:2002.05202, 2020. 3
- [11] Biao Zhang and Rico Sennrich. Root mean square layer normalization. In *Advances in Neural Information Processing Systems*, volume 32, pages 12360–12371, 2019. 3
- [12] Federico Berto, Chuanbo Hua, Junyoung Park, Laurin Luttman, Yining Ma, Fanchen Bu, Jiarui Wang, Haoran Ye, Minsu Kim, Sanghyeok Choi, Nayeli Gast Zepeda, André Hot-tung, Jianan Zhou, Jieyi Bi, Yu Hu, Fei Liu, Hyeonah Kim, Jiwoo Son, Haeyeon Kim, Davide Angioni, Wouter Kool, Zhiguang Cao, Qingfu Zhang, Joungho Kim, Jie Zhang, Kijung Shin, Cathy Wu, Sungsoo Ahn, Guojie Song, Changhyun Kwon, Kevin Tierney, Lin Xie, and Jinkyoo Park. RL4CO: an extensive reinforcement learning for combinatorial optimization benchmark. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 5278–5289, New York, NY, USA, 2025. doi: 10.1145/3711896.3737433. 3
- [13] Niels A. Wouda, Leon Lan, and Wouter Kool. PyVRP: A high-performance VRP solver package. *INFORMS Journal on Computing*, 36(4):943–955, 2024. doi: 10.1287/ijoc.2023.0055. 3
- [14] Amin Sghaier, Ali Parviz, and Alexia Jolicoeur-Martineau. Probabilistic tiny recursive model. arXiv preprint arXiv:2605.19943, 2026. 3
- [15] Oriol Vinyals, Meire Fortunato, and Navdeep Jaitly. Pointer networks. In *Advances in Neural Information Processing Systems*, volume 28, pages 2692–2700, 2015. 4